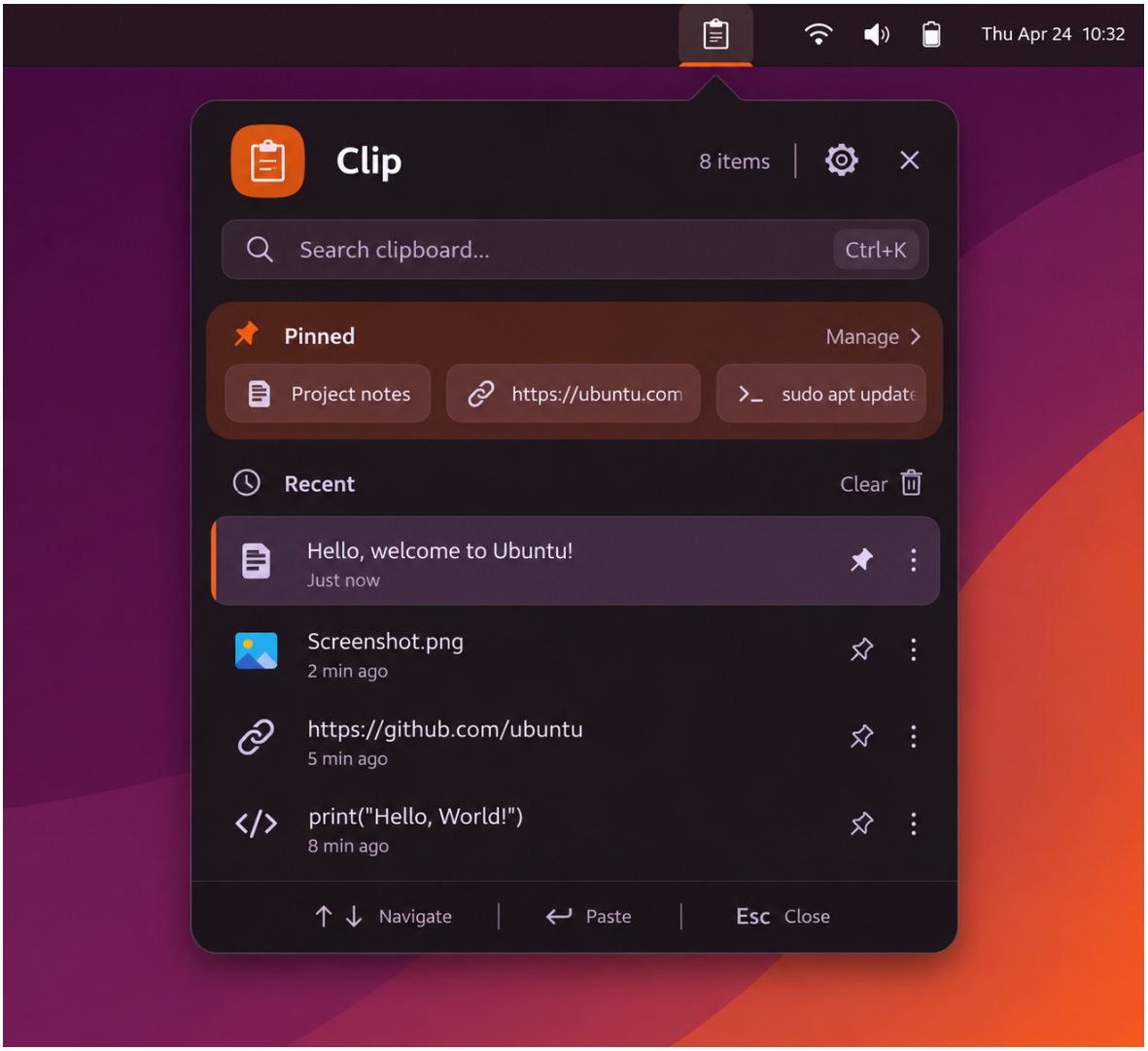
UBUNTU CLIP

Product Design, Technical Architecture & Implementation Plan

Implementation blueprint · v1.0 · 1 November 2026



A keyboard-first, Ubuntu-native clipboard manager designed around fast search, pinned clips, durable storage, and verified recovery.

Executive summary

Ubuntu Clip is a compact desktop utility rather than a heavyweight dashboard. Opening the configured shortcut immediately focuses search. Pinned clips live in a persistent shelf, recent items appear in a simple list, and Enter restores the selected item to the system clipboard. Automated paste is attempted only where the desktop integration explicitly supports it.

Reliability is first-class: a Rust service owns capture and persistence; SQLite uses WAL with synchronous=FULL; large blobs are written and synced before database references are committed; and backups are consistent, verified bundles containing the database and referenced blobs.

Platform caveat: Wayland clipboard-manager access and synthetic paste are not universally portable. Prove support on the exact Ubuntu/GNOME release before committing to the backend. Always retain a safe copy-to-clipboard fallback.

Contents

• 1. Product vision

• 2. Final UX and interaction specification

• 3. Visual design specification

• 4. Architecture

• 5. Storage model and crash-safe writes

• 6. Backup and restore strategy

• 7. Privacy and security

• 8. Failure matrix and expected behavior

• 9. Implementation roadmap and acceptance criteria

• 10. Suggested repository structure

• 11. Decisions to settle before coding

1. Product vision

Goal

Build a small, native-feeling Ubuntu clipboard manager: instant to open, immediately ready to search, keyboard-first, and resilient to application crashes, desktop restarts, and unexpected power loss.

Product principles

Fast before feature-rich; pinned items are always easy to reach; search is focused on open; storage is durable before an item is reported as saved; UI, clipboard capture, and persistence are separate components; privacy is explicit and local-first.

Initial platform target

Start with Ubuntu GNOME on one explicitly selected release and session type. Treat X11 and Wayland as separate integration targets with capability detection and a tested support matrix. Do not claim universal Wayland compatibility until verified on each target compositor.

2. Final UX and interaction specification

Popup

A compact Yaru-inspired popup anchored to a panel indicator. Adaptive light/dark styling follows the desktop. The header contains the Clip icon/name, item count, and settings. Search is the first prominent control and receives keyboard focus immediately when the popup opens.

Pinned shelf

A persistent horizontal shelf directly below search. Each item is a compact chip with a small useful icon and a truncated title/preview. It scrolls horizontally when necessary. Pinning is not a separate mode and pinned items remain searchable.

Recent list

A chronological, compact list with one or two preview lines, relative timestamp, and small format icon only when useful. Hover/focus reveals pin and overflow actions. Selected row has a subtle Ubuntu-orange leading accent.

Search behavior

On open, focus the search field and select any existing query text. Typing filters pinned and recent items immediately. Search includes text and metadata; image entries can be found by filename/description where available. Empty query shows pinned shelf plus recent history.

Selection and paste

Enter restores the selected item to the system clipboard, then requests paste only through a supported desktop integration. If automated paste is unavailable or fails, close the popup with the selected item already on the clipboard and show a brief 'Ready to paste — Ctrl+V' hint. Never silently claim paste succeeded.

Toggle and escape

The configured global shortcut toggles the popup. A second press closes it. Escape first clears a non-empty query; a subsequent Escape closes the popup. Clicking outside closes it. Panel indicator click also toggles.

Recommended default shortcut

Super+Shift+V, configurable. Do not assume Super+V is free: GNOME commonly assigns it to the notification/calendar panel. Register through the supported Global Shortcuts portal where available; use a GNOME Shell extension fallback only when necessary and permitted.

Additional shortcuts

Ctrl+K focuses search; Up/Down navigates; Enter restores/pastes; Ctrl+P pins/unpins; Delete removes selected history item; Ctrl+1…9 selects pinned entries; Escape clears query then closes. Make every shortcut configurable and show conflicts clearly.

3. Visual design specification

Identity

Use Ubuntu Yaru as the visual base: restrained aubergine/neutral surfaces, Ubuntu orange for selection and focus, familiar GNOME spacing and typography. Avoid a generic web-dashboard look.

Density

Target a compact popup around 400–440 px wide, with height adapting to available screen space. Show a small number of recent rows before scrolling. Keep controls visually quiet.

Typography and icons

Use the system interface font and symbolic icons. Avoid large category badges. Use tiny icons only when they improve recognition (text, image, link, code). Do not rely on color alone to distinguish content.

Adaptive theme

Use Libadwaita color scheme preference and system accent where available. Verify contrast and focus indicators in both light and dark modes.

Accessibility

Full keyboard operation, visible focus, screen-reader labels, scalable text, reduced-motion behavior, and adequate contrast. Avoid hover-only access to essential actions.

4. Architecture

Architecture overview

A thin GNOME integration layer controls the panel indicator and shortcut; a GTK4/Libadwaita desktop UI presents the popup; a Rust background service owns clipboard capture, indexing, storage, backup, and recovery. UI and extension communicate with the service over a versioned D-Bus API.

GNOME integration

Preferred path on compatible GNOME versions: Global Shortcuts portal for registering the user-approved shortcut. A small GNOME Shell extension may provide a native panel indicator and popup anchoring. Keep extension logic thin: no database access, no history ownership, no heavy work in Shell.

Clipboard adapters

Define a ClipboardBackend interface with capability detection. Implement and test a GNOME/Wayland backend for the chosen Ubuntu target, plus an X11 backend. Wayland clipboard-manager access is compositor/protocol-dependent; do not assume one

universal API. If history capture is unsupported, clearly report the limitation rather than pretending to capture.

Rust service

Own capture lifecycle, MIME/format normalization, size limits, hashing, deduplication, search, retention, persistence, backup, recovery, and structured logs. Keep all disk I/O off the UI thread.

IPC

Use D-Bus with typed methods/signals, explicit API versioning, request timeouts, reconnect handling, and bounded payload sizes. Transfer large content through files or file descriptors where supported rather than sending huge D-Bus messages.

Storage

SQLite for metadata and small text payloads; content-addressed blob files for larger images and binary payloads. Store under XDG_DATA_HOME/ubuntu-clip by default. Use XDG_CONFIG_HOME for preferences and XDG_STATE_HOME for logs/state where appropriate.

Service supervision

Run as a systemd user service with Restart=on-failure, start-rate limiting, structured journal logs, graceful shutdown, and a health/status endpoint over D-Bus. The UI should reconnect after service restart.

5. Storage model and crash-safe writes

Clipboard item record

Suggested fields: id (UUID), created_at, updated_at, mime_type, normalized_kind, preview, text_content (nullable), blob_hash (nullable), byte_size, content_hash, source_app (optional), pinned, sensitive, capture_status, schema_version.

Blob layout

blobs/sha256/<first-two-hex>/<full-hash>.blob. Write to a unique temporary file in the same filesystem, stream bytes, fsync the file, atomically rename to the final hash path, and fsync the parent directory where supported. Validate hash and size before committing the database reference.

Commit ordering

1) Capture and validate. 2) Normalize and hash. 3) Persist large blob to temp file. 4) fsync and atomically rename blob. 5) Commit SQLite metadata/reference transaction. 6) Emit Saved event to UI. If the process dies between steps 4 and 5, startup garbage collection finds an unreferenced blob. Never commit a DB reference before the blob is durable.

SQLite settings

Use WAL mode, foreign_keys=ON, a busy timeout, and synchronous=FULL for the strongest practical durability target. Benchmark on target hardware, but do not silently downgrade durability. FULL still depends on the filesystem, storage device, and correct fsync behavior.

Transactions

Use one serialized writer task/connection for mutations; allow separate read connections. Keep transactions short. Enforce unique content hashes for deduplication where appropriate. Avoid storing large binary payloads in the DB.

Recovery checks

On startup: acquire a single-instance lock; open DB; run quick_check (and periodic integrity_check); validate schema; inspect pending temp files; verify referenced blobs; identify orphan blobs; restore missing data only from verified backup; write a recovery report. Never silently delete uncertain data.

Disk-full behavior

If a write fails with ENOSPC or quota errors, do not mark the clip saved. Preserve existing data, stop or pause new capture with a clear status, prune only according to the configured retention policy, and notify the user.

Memory loss

The application must not depend on an in-memory queue for committed history. A small bounded queue is acceptable for capture throughput, but acknowledge a capture as durable only after disk commit. Data still in the OS clipboard but not yet captured cannot be guaranteed after a crash.

6. Backup and restore strategy

Backup format

A backup is a versioned bundle: consistent SQLite snapshot, all referenced blobs, manifest.json, checksums, application/schema version, creation timestamp, and optional encrypted archive metadata.

Database snapshot

Use SQLite Online Backup API or another documented consistent snapshot method. Do not copy a live .db file alone while WAL is active. The Online Backup API creates a consistent snapshot while allowing normal DB activity with brief read locks.

Snapshot schedule

Recommended initial defaults: daily verified snapshot; before application upgrades and schema migrations; user-triggered backup; optional periodic snapshot based on activity. Clipboard capture itself is already committed to primary storage, so backups are a second recovery layer, not the primary save path.

Retention

Keep a configurable rolling set, for example 7 daily and 4 weekly snapshots, with a maximum storage budget. Never prune the last known-good snapshot before a newer snapshot has passed verification.

Restore workflow

Select backup → verify manifest and checksums → extract into a staging directory → validate DB and every referenced blob → preserve current data as a pre-restore snapshot → atomically switch active data directory (same filesystem) → restart service → run post-restore verification. If any step fails, retain the old active data.

External backup

Offer an optional user-selected external drive or folder. Local snapshots do not protect against disk failure, theft, or filesystem loss. Clearly display last successful external backup and destination.

Encryption

Optional encrypted backups using a well-reviewed system library/format and a user-controlled passphrase or OS keyring. Do not invent custom cryptography. Explain that losing the key means losing access to encrypted backups.

7. Privacy and security

Sensitive content

Provide app exclusion rules, a pause-capture control, configurable history expiration, per-item deletion, and a sensitive-content policy. Avoid storing password-manager clipboard contents where detectable. Treat detection as best-effort, not perfect.

Local-first

No cloud upload or telemetry by default. Logs must never contain clipboard text, image bytes, or secrets. Use restrictive file permissions for data and backup directories.

Retention

Initial suggested limit: 1,000 entries plus configurable maximum storage size and automatic age expiration. Users can change limits. Avoid deleting pinned entries automatically unless explicitly configured.

Threat model

Clipboard data can contain credentials and personal information. Any process running as the same user may have broad access to local user data; file permissions and optional encryption reduce exposure but do not make the clipboard manager a secure vault.

8. Failure matrix and expected behavior

UI crash

Service remains running; reopen UI and reload state from DB.

Service crash

systemd restarts it; UI reconnects; committed records remain.

GNOME Shell restart

Extension reloads and reconnects; service/database remain independent.

Power loss / OS crash

SQLite WAL recovery plus filesystem-synced blob writes; last uncommitted capture may be lost, committed records should remain subject to hardware/filesystem guarantees.

Interrupted blob write

Temporary file is discarded or resumed only after validation; no DB reference exists.

Corrupt database

Quarantine current DB, attempt recovery/verified backup, preserve damaged files for diagnostics; never overwrite the only copy.

Missing/corrupt blob

Mark the affected entry unavailable, report it, and attempt recovery from a verified snapshot.

Upgrade interrupted

Keep pre-upgrade snapshot and schema migration marker; resume or roll back safely.

Disk full / permission denied

Pause capture, retain old history, show actionable warning, and do not claim new items were saved.

9. Implementation roadmap and acceptance criteria

Phase 0 — Platform spike

Identify exact Ubuntu release, GNOME version, Wayland/X11 session. Prove clipboard capture, panel integration, global shortcut registration, and paste behavior before committing to a backend. Deliver a short compatibility report.

Phase 1 — Storage core

Implement Rust domain model, SQLite schema/migrations, blob store, hashing/deduplication, transactional writer, retention, and unit tests. Acceptance: captured text survives service termination and reboot.

Phase 2 — Clipboard backend

Implement capability-based backend for target session; MIME normalization; capture retry; size limits; app exclusions; clipboard ownership and race handling. Acceptance: capture/restore tests across supported apps.

Phase 3 — UI

GTK4/Libadwaita popup, search focused on open, pinned shelf, recent rows, keyboard navigation, adaptive theme, accessibility. Acceptance: open/search/select/restore without mouse.

Phase 4 — GNOME integration

Panel indicator, shortcut registration, toggle behavior, outside-click close, D-Bus reconnect. Acceptance: no conflict with default GNOME shortcuts; Shell restart does not lose history.

Phase 5 — Backup/recovery

Online DB snapshots, blob manifests/checksums, retention, staged restore, startup verification and recovery report. Acceptance: restore from backup after deliberately corrupted DB and missing blob tests.

Phase 6 — Hardening and packaging

systemd user unit, logs, settings, installer/package, upgrade migrations, docs, release CI. Acceptance: install/upgrade/uninstall/reinstall tests and supported-session matrix.

Release gates

Run 10,000-entry stress test; repeated kill during capture; forced termination during DB commit; interrupted blob writes; disk-full simulation; interrupted backup/restore; DB integrity checks; missing/corrupt blob tests; Shell restart; Wayland/X11 matrix; large-image and duplicate tests.

10. Suggested repository structure

Workspace

Use a Rust Cargo workspace for the daemon and reusable crates; keep GTK UI and GNOME Shell extension separate. Version the D-Bus API and the on-disk schema independently.

Tree

ubuntu-clip/\n apps/desktop-ui/\n services/clipboard-daemon/\n integrations/gnome-extension/\n crates/clipboard-core/\n crates/storage/\n crates/search/\n crates/backup/\n crates/recovery/\n crates/ipc/\n tests/crash-recovery/\n tests/backup-restore/\n tests/platform/\n packaging/\n docs/

11. Decisions to settle before coding

Required

Choose the exact Ubuntu release and GNOME version; confirm Wayland or X11 as the first supported session; choose minimum supported GNOME version; decide whether panel integration is mandatory in v1 or whether a standalone popup is acceptable.

Recommended MVP

Target one current Ubuntu GNOME release first. Implement reliable text capture, pinned items, search, restore-to-clipboard, SQLite durability, and verified local backup. Add image previews and automated paste only after the base path is proven.

Explicit non-goals for v1

Cloud sync, collaboration, OCR, AI summaries, clipboard sharing across devices, universal Wayland support, and guaranteed automated paste into every application.

Appendix A — Core database schema (starting point)

```
CREATE TABLE clips (
id TEXT PRIMARY KEY,
created_at INTEGER NOT NULL,
mime_type TEXT NOT NULL,
kind TEXT NOT NULL,
preview TEXT,
text_content TEXT,
blob_hash TEXT,
byte_size INTEGER NOT NULL,
content_hash TEXT NOT NULL,
source_app TEXT,
pinned INTEGER NOT NULL DEFAULT 0,
sensitive INTEGER NOT NULL DEFAULT 0,
UNIQUE(content_hash)
);
CREATE INDEX clips_created_at_idx ON clips(created_at DESC);
CREATE INDEX clips_pinned_created_idx ON clips(pinned, created_at DESC);
-- Add FTS5 virtual table for searchable text and metadata.
```

Appendix B — Example user service unit

```
[Unit]
Description=Ubuntu Clip clipboard service
After=graphical-session.target

[Service]
Type=dbus
BusName=org.example.UbuntuClip
ExecStart=/usr/bin/ubuntu-clip-daemon
Restart=on-failure
RestartSec=2
StartLimitIntervalSec=60
StartLimitBurst=5

[Install]
WantedBy=default.target
```

Appendix C — Reference documentation

- **GNOME 48 developer notes — Global Shortcuts portal** — https://release.gnome.org/48/developers/
- **GNOME Shell extensions — overview and installation** — https://help.gnome.org/system-admin-guide/extensions.html
- **SQLite WAL documentation** — https://www.sqlite.org/wal.html
- **SQLite PRAGMA synchronous documentation** — https://www.sqlite.org/pragma.html
- **SQLite Online Backup API** — https://www.sqlite.org/backup.html
- **systemd.service — Restart= behavior** — https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html